

Chapitre 8

JDBC

Introduction

- ❑ JDBC (Java DataBase Connector) est une API chargée de communiquer avec les bases de données en Java.
- ❑ JDBC permet d'interagir avec les base de données de la même façon quelque soit leur fournisseur (Oracle, SQL Server, MySQL, PostgreSQL,...).
- ❑ Les classes et interfaces de l'API JDBC figurent dans le package java.sql :
`import java.sql.*;`
- ❑ JDBC peut être utilisé pour accéder à n'importe quelle base de données à partir d'une Simple application Java, d'une servlet, JSP,...

Connecteurs ou pilotes

- ❑ Pour communiquer avec une BDD, il suffit de télécharger la bibliothèque qui assure la communication entre Java et cette base de donnée.
- ❑ Cette bibliothèque s'appelle **Driver** ou **Pilote** ou **Connecteur**.
- ❑ Elle figure sur le site du fournisseur du SGBDR utilisé.
- ❑ Le connecteur MySQL pour Java se nomme comme cet exemple : "**mysql-connector-java-8.0.27.jar**"
- ❑ Ce connecteur doit être référencé par votre projet Java pour qu'il puisse communiquer avec votre BDD

Connecteurs ou pilotes

- ❑ Après téléchargement du connecteur il faut le stocker dans un dossier « **lib** » du projet avant de le référencer.
- ❑ **Eclipse :**
 - clic droit sur le projet → Properties → Java Build Path → Libraries → Add External JARs → sélectionner le connecteur JDBC (.jar) → Apply and Close.
- ❑ **NetBeans :**
 - clic droit sur le projet → Properties → Libraries → Add
- ❑ **IntelliJ IDEA :**
 - clic droit sur le projet → Open Module Settings (F4) → Modules → Dependencies → + → JARs or Directories → choisir le connecteur JDBC (.jar) → Apply → OK.JAR/Folder → sélectionner le connecteur JDBC (.jar) → OK.

Etapes d'interaction avec une BDD

1. Chargement du pilote ou connecteur
2. Etablissement de la connexion
3. Création des objets encapsulant les requêtes
4. Exécution des requêtes
5. Parcours des résultats dans le cas d'une requête de sélection
6. Fermeture des objets résultats, requêtes et connexion

Chargement du pilote

- ❑ Dans ce cours nous allons prendre MySQL comme exemple.
- ❑ Connecteur MySQL pour Java " **mysql-connector-java-x.x.xx.jar**"
- ❑ Pour se connecter à une base de données il faut charger son pilote.
- ❑ Chaque pilote dispose d'une classe principale que JDBC doit connaître
- ❑ La documentation de la Bdd utilisée fournit le nom de la classe à utiliser.
- ❑ Dans le cas de de cette version de Driver la classe est :
`com.mysql.cj.jdbc.Driver`

Chargement du pilote

□ Avant : JDBC 4.0

- Le chargement se faisait comme suit : `Class.forName(nom_classe_connecteur);`
- Exemple :
 - Dans le cas de la Bdd MySQL, ce chargement est comme suit :
`Class.forName(com.mysql.cj.jdbc.Driver)`
- Une fois chargée, la classe JDBC qui se nomme **DriverManager** prend en charge le driver pour communiquer avec la base de données.

Chargement du pilote

□ A partir de JDBC 4.0,

- Le chargement du pilote est automatique grâce au mécanisme de Service Provider introduit dans Java.
- Chaque driver JDBC moderne (comme MySQL, PostgreSQL, Oracle, etc.) contient un fichier de configuration : **META-INF/services/java.sql.Driver**
- Ce fichier indique à la JVM quelle classe de driver doit être chargée automatiquement au démarrage.
- **Résultat** : Dès que le JAR du driver est dans le classpath, le driver est détecté et enregistré automatiquement.

Classes de l'API JDBC

- Les classes et interfaces les plus usuelles sont les suivantes:
 - **DriverManager** (classe): charge et configure le driver de la base de données.
 - **Connection** (interface): réalise la connexion et l'authentification à la base de données.
 - **Statement** (interface): contient la requête SQL et la transmet à la base de données.
 - **PreparedStatement** (interface): représente une requête paramétrée
 - **ResultSet** (interface): représente les résultat d'une requête de sélection.

Etablissement de la connexion

- Pour se connecter à une base de données, il faut disposer d'un objet **Connection** créé grâce au DriverManager en lui passant :
 - l'URL de la base à accéder , Le login, Le mot de passe
- Exemple avec MySQL:
 - `String url="jdbc:mysql://localhost/mydb";`
 - `String login="root";`
 - `String password="motdepasse";`
 - `Connection con=DriverManager.getConnection(url, login, password);`

Etablissement de la connexion

- ❑ Exemples de classes de Drivers et d'URL de différentes bases de données

BDD	Classe du Driver	Exemple URL de connexion
MySQL	com.mysql.cj.jdbc.Driver	jdbc:mysql://localhost:3306/nom_base
MariaDB	org.mariadb.jdbc.Driver	jdbc:mariadb://localhost:3306/nom_base
PostgreSQL	org.postgresql.Driver	jdbc:postgresql://localhost:5432/nom_base
Oracle Database	oracle.jdbc.driver.OracleDriver	jdbc:oracle:thin:@localhost:1521:xe
SQL Server	com.microsoft.sqlserver.jdbc.SQLServerDriver	jdbc:sqlserver://localhost:1433;databaseName=nom_base
SQLite	org.sqlite.JDBC	jdbc:sqlite:chemin/vers/base.db

Exécution de requêtes SQL

- ❑ L'interface **Statement** permet d'envoyer des requêtes SQL à la base de données.
- ❑ Un objet Statement est créé de la façon suivante :
 - **Statement** st = **con.createStatement()**;
- ❑ Il possède deux méthodes :
 - **executeUpdate()** : Insertion, suppression, mise à jour.
 - ❑ int n= st.executeUpdate("**INSERT INTO Etudiant VALUES (3452,'Taha','Ali')**");
 - **executeQuery()** : Selection.
 - ❑ **ResultSet** res= st.**executeQuery("SELECT * FROM Etudiant")**;

Requêtes avec paramètres

- ❑ L'interface **PreparedStatement** permet d'envoyer des requêtes SQL à la base de données en prenant des paramètres.
- ❑ Ces paramètres sont représentés par des points d'interrogation(?) et doivent être spécifiés avant l'exécution.
- ❑ Exemple :

```
PreparedStatement p= con.prepareStatement("select* from Etudiant where cne=? And nom= ?");  
p.setInt(1, 3452345);  
p.setString(2, "Alaoui");  
ResultSet resultats = p.executeQuery();
```

Résultat d'une requête de sélection

- ❑ Une requête de sélection retourne un **ResultSet**
- ❑ ResultSet est un ensemble d'enregistrements constitués de colonnes qui contiennent les données.
- ❑ Les principales méthodes :
 - **next()** : se déplace sur le prochain enregistrement : retourne false si la fin est atteinte. Le curseur pointe initialement juste avant le premier enregistrement.
 - **getString(int/String)** : retourne le contenu de la colonne dont le numéro (resp. le nom) est passé en paramètre sous forme d'une chaîne de caractère.
 - **getInt(int/String)** : retourne le contenu de la colonne dont le numéro (resp. le nom) est passé en paramètre sous forme d'entier.

Résultat d'une requête de sélection

- **getFloat(int/String)** : retourne le contenu de la colonne sous forme de nombre flottant.
- **getDate(int/String)** : retourne le contenu de la colonne sous forme de date.
- **Close()** : ferme le ResultSet

Résultat d'une requête de sélection

❑ Exemple :

```
ResultSet res= st.executeQuery("SELECT * FROM Etudiant");
while (res.next()) {
    System.out.println("CNE= "+res.getString(1)+" Nom= " + res.getString(2)
                        +" Prénom= "+res.getString(3));
}
```


Exemple complet

```
import java.sql.*;
public class Main {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/etudiantsDB"; // Port par défaut 3306
        String user = "root";
        String password = "";
        String sql = "SELECT cne, nom, prenom FROM Etudiant";

        // Bloc try-with-resources = fermeture automatique de Connection, Statement, ResultSet
        try (
            Connection con = DriverManager.getConnection(url, user, password);
            Statement st = con.createStatement();
            ResultSet res = st.executeQuery(sql);
        ) {
            System.out.println("Connexion réussie à la base de données !");
            System.out.println("Liste des étudiants :");
            System.out.println("-----");
        }
    }
}
```

Exemple complet

```
// Parcours du ResultSet
while (res.next()) {
    String cne = res.getString("cne");
    String nom = res.getString("nom");
    String prenom = res.getString("prenom");

    System.out.println("CNE = " + cne + " | Nom = " + nom + " | Prénom = " + prenom);
}

} catch (Exception e) {
    e.printStackTrace();
}
}
```

Métadonnées sur la base de données

❑ Classes pour obtenir des métadonnées:

- **DatabaseMetaData** : informations à propos de la base de données : nom des tables, index, version ...
- **ResultSetMetaData** : informations sur les colonnes (nom et type) d'un ResultSet

❑ Exemple :

```
ResultSetMetaData meta = res.getMetaData();
int nbCols = meta.getColumnCount();
while (res.next()) {
    for (int i = 1; i <= nbCols; i++) {
        System.out.print(meta.getColumnName(i) + " = " + res.getString(i) + " ");
    }
    System.out.println();
}
```

Transactions

- ❑ Une transaction est un ensemble de requêtes qui doivent s'exécuter d'un seul bloc.
- ❑ Si une requête de cet ensemble échoue alors toutes les autres sont annulées
- ❑ Par contre si toutes les requêtes réussissent alors l'ensemble est validé.

Validation automatique

- ❑ Par défaut, les connexions sont en mode de validation automatique (auto-commit) où chaque instruction SQL est considérée comme une transaction.
- ❑ La validation automatique peut être désactivée grâce à la méthode `setAutoCommit()`
- ❑ Exemple :
 - `conn.setAutoCommit(false);` // `conn` est un objet `Connection`

Commit & Rollback

- ❑ Une fois les modifications sont terminées, ils sont validées grâce à la méthode **`commit()`** sur l'objet de connexion
- ❑ Exemple :
 - `conn.commit();`
- ❑ Si un problème se produit dans la suite des requêtes exécutées, alors l'ensemble peut être annulée grâce à la méthode `rollback()`
- ❑ Exemple :
 - `Conn.rollback()`

Commit & Rollback

❑ Exemple :

```
try{
    conn.setAutoCommit(false);
    Statement stmt = conn.createStatement();
    //requête correcte
    String requete1 = "INSERT INTO Etudiant VALUES (26, 'Alaoui', 'Ali')";
    stmt.executeUpdate(requete1);
    //requête fausse
    String requete2 = "INSERT INTOO Etudiant VALUES (27,'Omari', 'Omar')";
    stmt.executeUpdate(requete2);
    conn.commit(); // si il n y a pas d'erreur
}
catch(SQLException se){
    conn.rollback(); // si il y a des erreurs
}
```

Java - Dr A. Belangour

25

Points de sauvegarde

- ❑ Un point de sauvegarde, est un point de restauration logique dans une transaction.
- ❑ Si une erreur se produit après un point de sauvegarde, la méthode de restauration peut:
 - Annuler soit toutes les modifications,
 - Annuler uniquement les modifications apportées après le point de sauvegarde.
- ❑ L'objet Connection permet de gérer les points de sauvegarde grace aux méthodes :
 - `setSavepoint(String savepointName)` - Définit un nouveau point de sauvegarde et renvoie également un objet Savepoint.
 - `releaseSavepoint(Savepoint savepointName)` - Supprime un point de sauvegarde.

Java - Dr A. Belangour

26

Points de sauvegarde

❑ Exemple :

```
try{
    conn.setAutoCommit(false);
    Statement stmt = conn.createStatement();
    //requête correcte
    String requete1 = "INSERT INTO Etudiant VALUES (26, 'Alaoui', 'Ali')";
    stmt.executeUpdate(requete1);
    // définition du point de sauvegarde
    Savepoint savepoint1 = conn.setSavepoint("Savepoint1");
    //requête fausse
    String requete2 = "INSERT INTOO Etudiant VALUES (27,'Omari', 'Omar')";
    stmt.executeUpdate(requete2);
    conn.commit(); // si il n y a pas d'erreur
}
catch(SQLException se){
    conn.rollback(savepoint1); // retour au point de sauvegarde en cas d'erreur
}
```

Java - Dr A. Belangour

27

Clés générées

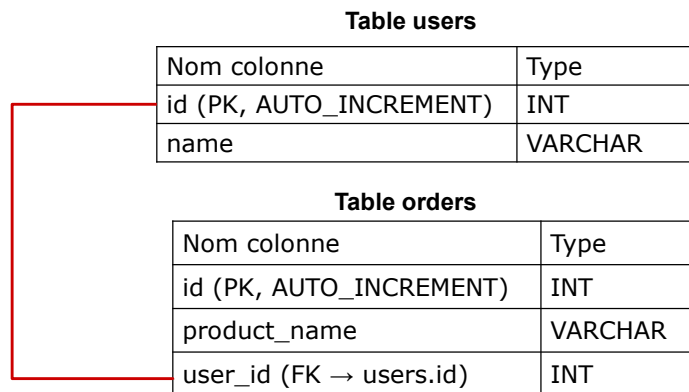
- ❑ Lors de l'insertion des données dans deux tables en relation, la clé principale d'une table est insérée comme clé étrangère dans l'autre table.
- ❑ Si cette clé est de type Auto-Increment alors il est possible de la récupérer par la méthode `getGeneratedKeys()` de l'interface `Statement`.
- ❑ Ces clés gnérées sont renvoyées sous forme d'un `ResultSet` :
 - `ResultSet resultSet = statement.getGeneratedKeys();`

cours JEE - Dr. Abdessamad Belangour

28

Clés générées : Exemple

- ❑ Soit une base de données « myDB » composée de deux tables « users » et « orders »



Clés générées : Exemple

```
import java.sql.*;
public class ExempleJDBC {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/myDB";
        String username = "root";
        String password = "";
        String sqlInsertUser = "INSERT INTO users (name) VALUES (?)";
        String sqlInsertOrder = "INSERT INTO orders (product_name, user_id) VALUES (?, ?)";

        try (Connection connection = DriverManager.getConnection(url, username, password)) {
            connection.setAutoCommit(false); // Début de la transaction
            try (PreparedStatement stmtUser = connection.prepareStatement(sqlInsertUser,
                                                                           Statement.RETURN_GENERATED_KEYS )){

                // --- Insertion d'un utilisateur ---
                stmtUser.setString(1, "Ali");
                stmtUser.executeUpdate();
```

Clés générées : Exemple

```
int userId;
try (ResultSet keys = stmtUser.getGeneratedKeys()) {
    if (keys.next()) { userId = keys.getInt(1); // récupération de l'ID auto-incrémenté
    } else { throw new SQLException("Aucune clé générée après l'insertion dans 'users'."); }
}
// --- Insertion d'une commande associée à cet utilisateur ---
try (PreparedStatement stmtOrder = connection.prepareStatement(sqlInsertOrder)) {
    stmtOrder.setString(1, "Laptop"); stmtOrder.setInt(2, userId); stmtOrder.executeUpdate();
}
connection.commit(); // Validation de la transaction
System.out.println("Insertion réussie. ID de l'utilisateur = " + userId);

} catch (SQLException ex) {
    connection.rollback(); System.err.println("Erreur détectée, rollback effectué.");ex.printStackTrace();
} finally { connection.setAutoCommit(true);}
} catch (SQLException e) { e.printStackTrace(); }
}
```